UNIVERSIDAD ORT
Uruguay

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel. 2902 15 05 Fax 2908 13 70
www.ort.edu.uy

EVALUACIÓN	Obligatorio	**GRUPO**	Todos	**FECHA**	09/Oct/2025
MATERIA	Diseño de Aplicaciones 2				
CARRERA					

CONDICIONES

- Puntaje máximo: 25 puntos
- Puntaje mínimo: 7 puntos
- Fecha de entrega: 20/Nov/2025 hasta las 21:00 horas en gestion.ort.edu.uy (max. 40Mb en formato zip, rar o pdf) y en GitHub.

Uso de material de apoyo y/o consulta

Inteligencia Artificial Generativa

- Seguir las pautas de los docentes: Se deben seguir las instrucciones específicas de los docentes sobre cómo utilizar la IA en cada curso.
- Citar correctamente las fuentes y usos de IA: Siempre que se utilice una herramienta de IA para generar contenido, se debe citar adecuadamente la fuente y la forma en que se utilizó.
- Verificar el contenido generado por la IA: No todo el contenido generado por la IA es correcto o preciso. Es esencial que los estudiantes verifiquen la información antes de usarla.
- Ser responsables con el uso de la IA: Conocer los riesgos y desafíos, como la creación de "alucinaciones", los peligros para la privacidad, las cuestiones de propiedad intelectual, los sesgos inherentes y la producción de contenido falso
- En caso de existir dudas sobre la autoría, plagio o uso no atribuido de IAG, el docente tendrá la opción de convocar al equipo de obligatorio a una defensa específica e individual sobre el tema

Defensa

Fecha de defensa: Entre semana 15 y 16 del curso
La defensa es obligatoria y eliminatoria. El docente es quien definirá y comunicará la modalidad, y mecánica de defensa. La no presentación a la misma implica la pérdida de la totalidad de los puntos del Obligatorio.

IMPORTANTE:

1. Inscribirse en la evaluación a través de gestion.ort.edu.uy.

2. Formar grupos de tres personas (no es necesario que pertenezcan al mismo dictado).

3. Subir el trabajo a Gestión (documentación) y a GitHub antes de la hora indicada (ver la sección "RECORDATORIO" al final del documento).

4. Dentro del repositorio privado en la ORG de DA2 de GitHub realizar un *merge* como *release* a *master* (GitFlow) antes de la hora indicada.

Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con el Coordinador de cursos o Coordinación adjunta **antes de las 20:00h** del día de la entrega, a través de los mails crosa@ort.edu.uy / posada_l@ort.edu.uy (matutino) / larrosa@ort.edu.uy (nocturno), o vía Ms Teams.

Obligatorio 2 de Diseño de Aplicaciones II

Como continuación del trabajo obligatorio, el presente documento amplía los requerimientos de la primera entrega. Los equipos deberán analizar el impacto de los siguientes requerimientos en la arquitectura y el diseño previamente implementado, justificando las decisiones de refactorización necesarias.

- **Implementación de interfaz de usuario (UI).**
  El objetivo principal de esta nueva versión es completar la interfaz de usuario (*frontend*). Para ello, se debe construir una aplicación Angular (SPA) que implemente **TODAS** las funcionalidades descritas en la letra de la entrega anterior y en esta.

- **Canje de recompensas.**
  Los visitantes podrán **canjear puntos por recompensas** dentro de la aplicación.
  Cada recompensa tiene: nombre, descripción, costo en puntos, cantidad disponible y nivel de membresía requerido (si aplica, pude no requerir un nivel específico).
  El sistema debe:
  - Dar de alta recompensas.
  - Validar el puntaje y la disponibilidad antes de confirmar el canje.
  - Registrar cada operación en un historial de canjes.
  - Descontar automáticamente los puntos del visitante y actualizar el ranking.

- **Gestión de mantenimiento preventivo.**
  Extendiendo el requerimiento existente de *Gestión de incidencias* de la primera entrega, el administrador podrá programar (dar de alta) **mantenimientos preventivos** sobre las atracciones.
  Cada mantenimiento incluye: fecha, hora de inicio, duración estimada y descripción.
  Durante el mantenimiento:
  - La atracción no podrá recibir visitantes ni nuevas reservas.
  - Se generará automáticamente una incidencia temporal asociada al mantenimiento.

- **Historial de puntuación.**
  Cada visitante podrá consultar su **historial de puntuación**, mostrando para cada registro:
  - Fecha y hora.
  - Origen de los puntos (atracción, misión, evento especial, canje, etc.).
  - Estrategia de puntaje activa en el momento.

  Este historial servirá como auditoría y trazabilidad del sistema de puntuación.

- **Carga dinámica de estrategias de puntaje.**
  El sistema deberá permitir **incorporar nuevas estrategias de puntaje de forma dinámica y en tiempo de ejecución**.
  Para evitar acoplamiento y permitir extensibilidad futura, se debe diseñar un mecanismo de plugin mediante *reflection* que soporte la carga de nuevas estrategias desde ensamblados externos (*dll's*).
  El sistema debe permitir que terceros puedan definir y agregar nuevas estrategias sin necesidad de recompilar la solución.

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel. 2902 15 05 Fax 2908 13 70
www.ort.edu.uy

El sistema explorará los *assemblies* ubicados en una carpeta de plugins (/Plugins), y cargará todas las estrategias válidas. El usuario podrá elegir una estrategia desde un menú dinámico en la UI.

Como no se conocen todas las estrategias posibles y se quiere permitir extensibilidad a futuro, es que se desea que un tercero pueda extender nuestro sistema agregando nuevas estrategias. **Estas opciones deben poder ser agregadas al sistema en tiempo de ejecución sin necesidad de recompilar la aplicación.**

Se debe definir y documentar la/las interfaces, asumiendo que otro desarrollador la va a utilizar para desarrollar una nueva estrategia de puntaje, explicando claramente como utilizarla para dejar nuevas estrategias dentro de la aplicación. Se debe cumplir además con lo siguiente:

- La entrega debe incluir documentación detallada de los mecanismos de extensibilidad incorporados.
- La interfaz gráfica del sistema debe estar diseñada y preparada para trabajar en base a estos mecanismos, de forma de no sufrir cambios para poder agregar nueva funcionalidad.
- Se debe entregar todos los elementos compilados para una **nueva estrategia de puntaje:**
    - PuntuacionPorHora: multiplica puntos durante ciertas horas del día.
- **No** es necesario pasar las tres estrategias de puntaje de la primera entrega a que sean cargadas dinámicamente.

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel. 2902 15 05 Fax 2908 13 70
www.ort.edu.uy

Alcance

Se deben realizar todos los cambios solicitados, así como dejar bien documentado el impacto de cada uno de ellos en la implementación (qué paquetes/clases se impactó, qué se agregó o eliminó, qué modificaciones se realizaron, etc.).

Implementación

Se mantienen los siguientes requisitos no funcionales de la primera versión:

- La entrega debe contener una solución que agrupe todos los proyectos implementados.
- La solución debe incluir el código de las pruebas automáticas. Se requiere escribir los casos de prueba automatizados con el mismo framework de pruebas unitarias del primer obligatorio, documentando y justificando las pruebas realizadas.
- Se espera que la aplicación se entregue con una base de datos con datos de prueba, de manera de poder comenzar las pruebas sin tener que definir una cantidad de datos iniciales. Dichos datos de prueba deben estar adecuadamente especificados en la documentación entregada.

Tecnologías y herramientas de desarrollo

- Microsoft VSCode/ Visual Studio
- Microsoft SQL Server Express 2017
- Postman
- .NET SDK 8.0/ ASP.NET 8.0 (C#)
- Entity Framework Core 8.0
- Astah o cualquier otra herramienta UML 2
- Angular

Instalación

El costo de instalación de la aplicación debe de ser mínimo y documentado adecuadamente. **Agregar esta guía de instalación en la carpeta "Datos" del repositorio.**

NOTA: La totalidad y detalle de los requisitos serán relevados a partir de consultas en el foro correspondiente en aulas. Para evitar complejidades innecesarias se realizaron simplificaciones al dominio del problema real.

Independencia de librerías

Se debe diseñar la solución que al modificar el código fuente minimice el impacto del cambio en los componentes físicos de la solución. Debe documentar explícitamente como su solución cumple con

este punto. Cada paquete lógico debe ser implementado en un *assembly* independiente, documentando cuáles de los elementos internos al paquete son públicos y cuáles privados, o sea cuáles son las interfaces de cada *assembly*.

Usabilidad

La aplicación debe ser fácil de utilizar, intuitiva y atractiva. Se deben tener las consideraciones básicas de usabilidad brindadas en otros cursos (ej: Heurísticas de Nielsen). Se aconseja y fomenta el uso de librerías de terceros como: Bootstrap, Angular Material, Nebular, etc.

Persistencia de los datos

La empresa requiere que todos los datos del sistema sean persistidos en una base de datos. De esta manera, la siguiente vez que se ejecute la aplicación se comenzará con dichos datos cargados con el último estado guardado antes de cerrar la sesión o aplicación.

Diseño: Informe de métricas

Para esta segunda entrega se debe tomar un enfoque de fábrica de software con respecto al diseño utilizado. Esto quiere decir que se **debe promover la mantenibilidad del sistema y analizar al máximo que componentes de la aplicación podrían ser reutilizados** en otras aplicaciones a construir a futuro, y diseñar e implementar en función de ello, justificando las decisiones tomadas.

Se pide: realizar un informe sobre el diseño presentado, **basado en métricas**, que explique los puntos fuertes del diseño y que aspectos podrían ser mejorados. Este informe debe analizar los valores obtenidos de las métricas en función de los principios fundamentales de diseño, analizando cuales se cumplen, cuáles no, que ventajas y desventajas presenta y qué se podría cambiar para mejorar.

Se recomienda utilizar alguna herramienta para calcular las métricas en función del código (por ejemplo, NDepend). Particularmente se espera que se usen las métricas vistas en el curso de: *H, A, I y D (o D').*

Persistencia en base de datos

Toda la información contenida en el sistema debe ser persistida en una base de datos. El diseño debe contemplar el modelado de una solución de persistencia adecuada para el problema utilizando Entity Framework Core (*Code First*).

Se espera que como parte de la entrega se incluya dos respaldos de la base de datos: uno vacío y otro con datos de prueba. Se debe entregar el script *.sql* para ambas bases de datos.

Es condición necesaria para obtener el puntaje mínimo del obligatorio, que al menos una entidad del sistema pueda ser persistida.

Mantenibilidad

La propia empresa eventualmente hará cambios sobre el sistema, por lo que se requiere un alto grado de mantenibilidad, flexibilidad, calidad, claridad del código y documentación adecuada. Por lo que el desarrollo de todo el obligatorio debe cumplir:

- Estar en un repositorio **Git**.
- Haber sido escrito utilizando **TDD** (desarrollo guiado por pruebas) lo que involucra otras dos prácticas: escribir las pruebas primero (Test First Development) y refactoring. De esta forma se utilizan las pruebas unitarias para dirigir el diseño.
- Es necesario utilizar **TDD únicamente** para el *backend* y la *API REST*, no para el desarrollo del *frontend*. Se debe utilizar un framework de Mocking (como Moq) para poder realizar pruebas unitarias sobre la lógica de negocio. En caso de necesitar hacer un test double del acceso a datos, podrán hacerlo utilizando el mismo framework de Mocking anterior o de lo contrario con el paquete EF Core Sqlite.
- Por último, cumplir los lineamientos de **Clean Code** (capítulos 1 al 10, y el 12), utilizando las técnicas y metodologías ágiles presentadas para crear código limpio.

Control de versiones

La gestión del código del obligatorio debe realizarse utilizando UN ÚNICO repositorio Git de **Github**, apoyándose en el flujo de trabajo recomendado por **GitFlow**. Dicho repositorio debe pertenecer a la organización de GitHub "**IngSoft-DA2**" (https://github.com/IngSoft-DA2, en la cual deben estar todos los miembros del equipo.

Al realizar la entrega se debe realizar un *release* en el repositorio y la rama *main* no debe ser afectada luego del hito que corresponde a la entrega del obligatorio.

Evaluación (25 puntos)

Las condiciones de entrega serán evaluadas como si se le estuviese entregando a un cliente real: prolijidad, claridad, profesionalismo, orden, etc.

La entrega debe realizarse antes de las 21:00 horas del día de la entrega, tanto en **gestion.ort.edu.uy** como en el repositorio de GitHub del equipo. A continuación, se detallan los requisitos para cada uno:

1. **Entrega en Gestión (gestion.ort.edu.uy)**:

- La documentación completa debe estar en formato PDF e incluir el modelado UML. Es fundamental que todos los diagramas sean claros y legibles.
- La documentación debe estar contenida **en un solo documento** teniendo en cuenta los ítems que se describen en el apartado de evaluación.

Todos los documentos deben cumplir con los siguientes elementos del Documento 302 de la facultad (http://www.ort.edu.uy/fi/pdf/documento302facultaddeingenieria.pdf):

- Capítulo 3, secciones 3.1 (sin la leyenda), 3.2, 3.5, 3.7, 3.8 y 3.9.
- Capítulo 4 (salvo 4.1).
- Capítulo 5.
- Se **debe incluir en la portada del documento la URL al repositorio del equipo.**
- **Se debe actualizar todos los diagramas que corresponda y agregar los que se considere necesario para mostrar los cambios realizados y justificar las decisiones tomadas. Para esta segunda entrega es necesario repetir la documentación de diseño de la primera entrega actualizada.**

2. **Entrega en el Repositorio GitHub del equipo**:

- **Carpeta raíz**: Contendrá el código fuente de la aplicación, junto con todo lo necesario para compilar y ejecutar la aplicación.
- **Carpeta "Aplicación"**: Incluirá la aplicación compilada en modo *release* y todos los elementos necesarios para realizar la instalación (*deploy*) de la misma.
- **Carpeta "Datos"**: Incluirá el instructivo de instalación y los archivos '*.sql*'. Se debe entregar una versión de la base de datos vacía y otra con datos de prueba.

Rúbrica de evaluación

Area	**Demo al cliente**
Puntaje	2
Evaluación (criterios)	Cada equipo deberá realizar una demostración al cliente de su trabajo, **en las computadoras de los laboratorios de la Universidad o de la forma que los docentes comuniquen para el día de la demo**. Dentro de los aspectos que un cliente espera se encuentran: ● **Ver la demo cuando él esté listo y no tener que esperar a que el equipo se apronte.** ● **Los datos y la funcionalidad para mostrar deben estar preparadas.** ● Una **duración máxima de 15 minutos** y en el cual se pueda apreciar claramente la ejecución de todas las pruebas. ● Probar la solución y que la misma funcione sin problemas o con problemas mínimos, y de buena calidad de interfaz de usuario.

	● Conocer las capacidades de cada uno de los integrantes del equipo, pudiendo preguntar a cualquier integrante sobre la solución, su diseño, el código y sobre cómo fue construida, y así apreciar que fue un trabajo en equipo. Todos los integrantes deben conocer toda la solución. ● Verificar el aporte individual al trabajo por parte de cada uno de los integrantes del equipo y en función de los resultados, se podrán otorgar distintas notas a los integrantes del grupo. Se espera que cada uno de los integrantes haya participado en la codificación de parte significativa del obligatorio. Esta demostración al cliente hará las veces de **defensa del trabajo**. NOTA: El incorrecto funcionamiento de la instalación puede significar la no corrección de la funcionalidad. En el caso de defensa en el laboratorio, durante la defensa cada grupo contará con 15 minutos para la instalación de la aplicación. Luego de transcurridos los mismos se restan puntos al trabajo.

Area	**Funcionalidad**
Puntaje	6
Evaluación (criterios)	La asignación de los puntos de este ítem se basará en los siguientes criterios de corrección (escala): ● Se implementaron sin errores todos los requerimientos funcionales propuestos. ● Se implementaron todos los requerimientos funcionales propuestos, pero se detectan errores menores que no afectan el uso normal del sistema. ● Se implementaron los principales requerimientos funcionales, aunque no todos, pero se detectan errores menores que no afectan el uso normal del sistema. ● Se implementaron los principales requerimientos funcionales, aunque no todos, pero se detectan errores que afectan el uso normal del sistema. ● Los requerimientos funcionales implementados son básicos y/o se detectan errores que afectan el uso normal del sistema.

Area	**Diseño y Documentación**
Puntaje	12
Evaluación (criterios)	**El objetivo de este documento es demostrar que el equipo fue capaz de mejorar el diseño realizado y mejorar la documentación de la solución**.

La documentación debe ser pensada para que un tercero (corrector) pueda en base a la misma comprender la estructura y los principales mecanismos que están presentes en el código. O sea, debe servir como guía para entender el código y los aspectos más relevantes del diseño y la implementación.

El documento debe organizarse siguiendo el modelo 4+1 haciendo énfasis en todas las vistas que conoce. Este documento no debe incluir paquetes o componentes de los proyectos de prueba.

Elementos a evaluar:

- **La documentación no debe superar las 25 páginas (sin incluir el anexo).**
- Descripción general del trabajo (qué hace la solución) y errores conocidos (*bugs* o funcionalidades no implementadas).
- Diagrama general de paquetes (namespaces) mostrando los paquetes organizados por capas (*layers*) y sus dependencias. En caso de que haya paquetes anidados, se debe utilizar el conector de *nesting*, mostrando la jerarquía de dichos paquetes.
- Cada paquete debe tener una breve descripción de responsabilidades y un diagrama de clases asociado.
- Descripción de jerarquías de herencia utilizadas (en caso de que así haya sido)
- Modelo de tablas de la estructura de la base de datos.
- Para aquellas funcionalidades que el equipo entienda como relevantes:
    - Diagramas de interacción que muestran las clases involucradas en estas funcionalidades y las interacciones para lograr la funcionalidad.
    - Estas interacciones a mostrar pueden ser del flujo a alto nivel, de un cierto algoritmo específico de su obligatorio, etc.
- Diagrama de implementación (componentes) mostrando las dependencias entre los mismos. Justificar el motivo por el cual se dividió la solución en dichos componentes.
- Diagrama de despliegue (entrega) mostrando la dependencia entre los nodos.
- Justificación y explicación del diseño en base al uso de principios de diseño, patrones de diseño y métricas.
- Se debe discutir claramente los mecanismos utilizados para permitir la **extensibilidad solicitada en la funcionalidad de este nuevo obligatorio.**
- Se debe explicar claramente el uso de los patrones y principios de diseño utilizados y que valor aportan a la solución **(es importante que la explicación se realice en base a diagramas).**

- **Documentar las decisiones de diseño adoptadas y describir los principales aspectos de la solución implementada en el frontend (Angular).**
- **Se debe analizar la calidad del diseño discutiendo la calidad de este en base a las métricas de diseño y contrastándolas respecto a la aplicación de principios.** Es importante discutir en base a las métricas el cumplimento de los principios de: Clausura Común, Reuso común, Abstracciones estables, Dependencias estables y hacer hincapié en las conclusiones que se desprenden de la métrica de distancia, explicando los resultados en base a la abstracción e inestabilidad.
- **Resumen de las mejoras al diseño – se debe describir brevemente <u>las cuatro principales mejoras o cambios</u> que realizó sobre el diseño de la primera entrega** de forma de demostrar cómo la aplicación de principios de diseño y el uso de patrones aportaron para obtener un diseño más mantenible.

Para el anexo:

- **Se debe incluir el documento de especificación de la API con sus cambios. Solo con incluir aquellos cambios de esta nueva versión está correcto.**
- **Informe de cobertura analizando la efectividad de las pruebas en base a los valores de cobertura obtenidos.**

La documentación en general se considera como aceptable si:

- Los diagramas presentan el uso adecuado de la notación UML.
- La estructura de la solución representa la descomposición lógica (módulos) y de componentes
- Las vistas de módulos, de componentes, modelo de datos y los comportamientos documentados sirven como guía para la comprensión del código implementado.
- La taxonomía de paquetes y clases en los diagramas respeta las convenciones de nombre de C# utilizada en la implementación.
- **Se justifica y explica el diseño en base al uso de principios a nivel de clase y paquetes/componentes y patrones de diseño**.
- **Se explica correctamente los patrones de diseño utilizados en la solución en base a diagramas de estructura y comportamiento.**
- **Se analiza el diseño en base a principios y métricas y se justifican los desvíos obtenidos respecto a los valores de referencia.**
- **Se describen claramente las principales mejoras realizadas sobre el diseño de la primera entrega.**

	● **Se analiza la cobertura lograda por las pruebas discutiendo en forma crítica la efectividad de las pruebas automatizadas.** ● Correcto manejo de las excepciones e implementación de acceso a base de datos. ● Se describen claramente los errores conocidos. ● **Se documentan las mejoras a la API en caso de haber realizado cambios.** ● La documentación se encuentra bien organizada, es fácil de leer y su formato corresponde con los ítems indicados del documento 302.

Area	**Implementación**
Puntaje	5
Evaluación (criterios)	Se espera ver: ● Correcta aplicación de desarrollo guiado por las pruebas (TDD) y técnicas de refactoring de código. ● Utilización de buenas prácticas de estilo y codificación y su impacto en la mantenibilidad (Clean Code). ● Correcto uso de las tecnologías. ● Claridad del código respetando las guías de estilo de C#. ● Concordancia con el diseño documentado. ● Correcto manejo de excepciones. ● Implementación de acceso a base de datos. ● Queda explícitamente prohibido incorporar fragmentos de código generados automáticamente por herramientas de IA sin revisión, adaptación y justificación técnica propia. La autoría se evaluará tanto en la estructura del diseño como en las decisiones de implementación. El desarrollo del trabajo debe realizarse utilizando exclusivamente los conceptos, metodologías y tecnologías presentados en clase, aplicados de forma coherente e integrada en la solución. • Se perderán todos los puntos de esta sección si no se hace uso de Pull Request y de las Github Actions pactadas. Más información de configuración y uso: ○ Repositorio y Github Actions https://github.com/IngSoft-DA2/DA2-Tecnologia/tree/repo-configuration ○ Las Github Actions tienen que ser ejecutadas usando self hosted runners los cuales deben ser configuradas por cada integrante del equipo https://github.com/IngSoft-DA2/DA2-Tecnologia/blob/main/github-self-hosted-runner-docker.md ○ Pull Request: https://github.com/IngSoft-DA2/DA2-Tecnologia/tree/pr-creation

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel. 2902 15 05 Fax 2908 13 70
www.ort.edu.uy

RECORDATORIO: IMPORTANTE PARA LA ENTREGA

- **Obligatorios**

La entrega de los obligatorios será en formato digital online, a excepción de algunas materias que se entregarán en Bedelía y en ese caso recibirá información específica en el dictado de la misma.

Los principales aspectos a destacar sobre la **entrega online de obligatorios** son:

1. Ingresá al sistema de Gestión.
2. En el menú, seleccioná el ítem "Evaluaciones" y la instancia de evaluación correspondiente, que figura bajo el título "Inscripto".
3. Para iniciar la entrega hacé clic en el ícono:
4. Ingresá el número de estudiante de cada uno de los integrantes y hacé clic en "Agregar". El sistema confirmará que los integrantes estén inscriptos al obligatorio y, de ser así, mostrará el nombre y la fotografía de cada uno de ellos. Una vez agregados todos los integrantes, hacé clic en "Crear equipo".

   Cualquier integrante podrá:

   • Modificar la integración del equipo.

   • Subir el archivo de la entrega.
5. Seleccioná el archivo que deseás entregar. Verificá el nombre del archivo que aparecerá en la pantalla y hacé clic en "Subir" para iniciar la entrega. Cada equipo (de 3 estudiantes) debe entregar **un único archivo en formato zip o rar** (los documentos de texto deben ser pdf, y deben ir dentro del zip o rar). El archivo a subir debe tener **un tamaño máximo de 40mb**

   Cuando el archivo quede subido, se mostrará el nombre generado por el sistema (1), el tamaño y la fecha en que fue subido.
6. El sistema enviará un e-mail a todos los integrantes del equipo informando los detalles del archivo entregado y confirmando que la entrega fue realizada correctamente.
7. Podés cerrar la pestaña de entrega y continuar utilizando Gestión o salir del sistema.
8. La **hora tope para subir el archivo será las 21:00** del día fijado para la entrega.
9. La entrega se podrá realizar desde cualquier lugar (ej. hogar del estudiante, laboratorios de la Universidad, etc).
10. Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con el Coordinador de cursos o Coordinación adjunta antes de las 20:00h del día de la entrega, a través de los mails crosa@ort.edu.uy / posada_l@ort.edu.uy (matutino) / larrosa@ort.edu.uy (nocturno), o vía Ms Teams.